9. Detection, Segmentation, Visualization, and Understanding

1. Transformers Recap

세 가지 시퀀스 처리 방식

입력 사이의 정보를 어떤 방식으로 주고받는가?

방식	입력 구조	장점	한계
RNN	순서가 있는 1차원 시퀀스	길이 N에 대해 연산량 O(N)	hidden state를 순차적으로 계산 → 병렬화 어려움
Convolution	N차원 격자	여러 위치의 출력을 병렬 계산 가능	멀리 떨어진 입력을 연결하려면 여러 층 필요
Self-Attention	벡터들의 집합	각 출력이 모든 입력을 직접 참조, 행렬곱 중심으로 병렬화 가능	길이 N에 대해 연산량 $O(N^2)$

- **RNN**: 이전 hidden state를 받아 다음 hidden state 계산.
- **Convolution**: 주변의 제한된 영역을 보고 계산 → 깊게 쌓으면서 receptive field 확대.
- **Self-Attention**: 입력 간 관계를 직접 계산 → 먼 위치의 정보도 한 층에서 참조 가능.

Vision Transformer(ViT)

이미지에도 NLP의 Transformer를 적용 → 이미지를 패치 단위의 벡터로 변환.

1. **이미지→ patch 변환**
2. **patch를 D차원 vector로 projection**
   - 패치 하나를 펼치면 16 × 16 × 3 = **768차원**.
   - 학습 가능한 선형 변환으로 **768** → **D**.
3. **Positional embedding 추가**
   - 패치 벡터에 위치별 D차원 벡터를 더함.

- 각 패치가 이미지의 어디에서 왔는지 알려 주는 역할.

4. **Transformer에 입력**
   - 각 패치에 대응하는 출력 벡터를 얻음.
5. **Classification token으로 이미지 분류**
   - 패치 외에 **학습 가능한 D차원 토큰**을 추가.
   - 이 토큰에 대응하는 출력 → 선형 층 → C개 클래스 점수.

패치별 특징을 처리하면서, classification token의 출력으로 이미지 전체를 분류하는 구조.

다른 분류 방식 — Average Pooling

패치 변환과 Transformer 처리는 동일하고, 마지막 분류 방법이 달라짐.

- 위치 정보를 넣어 **패치의 2차원 위치**를 전달.
- **Masking 없음** → 각 패치는 다른 모든 패치를 볼 수 있음.
- Transformer 출력: 패치마다 D차원 벡터 → 전체 **N × D**.
- 출력 벡터들을 평균 → **1 × D**.
- 선형 층 **D** → **C** → 클래스 점수.

분류용 출력 구성 두 가지

방식	이미지 전체를 대표하는 벡터
Classification token	추가한 토큰의 출력
Average pooling	패치 출력 벡터들의 평균

---

2. Computer Vision Tasks

4가지 과제 구분

과제	무엇을 예측하나	같은 클래스의 개체 구분
Classification	이미지 전체의 클래스	구분하지 않음
Semantic Segmentation	각 픽셀의 클래스	구분하지 않음
Object Detection	각 객체의 클래스 + bounding box	구분함
Instance Segmentation	각 객체의 클래스 + 픽셀 마스크	구분함

- **Classification**: “고양이가 있다.”

- **Semantic Segmentation**: “이 픽셀은 고양이, 저 픽셀은 잔디.”
- **Object Detection**: “개 두 마리와 고양이 한 마리가 각각 이 상자 안에 있다.”
- **Instance Segmentation**: 각 객체가 차지하는 픽셀까지 구분.

2.1. Semantic Segmentation — 픽셀마다 분류

문제 정의

같은 클래스의 개체는 따로 구분하지 않음.

- **학습 데이터**: 이미지의 모든 픽셀에 semantic category가 붙어 있음.
- **추론**: 새로운 이미지의 각 픽셀이 어느 클래스인지 예측.

소가 여러 마리 있어도, 해당 픽셀들의 클래스는 모두 `Cow`.

첫 번째 아이디어 — Sliding Windo

픽셀 하나의 색만 보고 클래스를 판단할 수 있을까? → 주변 문맥이 필요.

- 픽셀 주변의 **patch**를 추출.
- patch를 CNN에 넣어 **중앙 픽셀의 클래스**를 예측.
- 이미지의 여러 위치에서 반복.

문제: 겹치는 patch 사이의 계산을 공유하지 못함.

서로 가까운 픽셀은 거의 같은 주변 영역을 보는데도 CNN을 따로 실행

→ 동일한 특징을 반복 계산 → 매우 비효율적.

두 번째 아이디어 — 전체 이미지를 CNN에 넣기

이미지 전체에서 특징을 한 번 추출하고, 그 특징으로 모든 픽셀을 예측.

문제 1: 일반적인 분류 CNN은 깊어지면서 공간 크기가 줄어듦.

- 분류: 이미지 전체를 대표하는 출력이 필요.
- 분할: 입력의 각 픽셀에 대응하는 출력이 필요.

해결 시도: downsampling 없이 convolution만 사용.

- C: 클래스 수.
- 각 위치에서 C개 클래스 점수를 얻음.

- 클래스 축으로 **argmax** → 해당 픽셀의 예측 클래스.

문제 2: 원본 해상도에서 convolution을 계속 계산하면 비용이 큼.

Downsampling + Upsampling

네트워크 안에서 해상도를 줄여 처리하고, 마지막에 다시 키움.

입력 → downsampling → 저해상도 특징 처리 → upsampling → 픽셀별 예측

- **Downsampling**: pooling, strided convolution.
- **Upsampling**: 공간 크기를 다시 확대하는 연산 필요.

⇒ 다음 질문: **작아진 feature map을 어떻게 키울까?**

Unpooling — 정해진 규칙으로 확대

방식	확대 방법
Nearest Neighbor	입력값을 주변 위치에 복제
Bed of Nails	입력값을 정해진 위치에 놓고, 나머지는 0

슬라이드 예시: **2 × 2** → **4 × 4**.

- Nearest Neighbor: 값 하나를 2 × 2 영역으로 복제.
- Bed of Nails: 값 사이에 0을 삽입.

Max Unpooling — pooling 위치를 기억하기

Max pooling 때 최댓값이 있던 위치를 저장해 두고, upsampling에서 재사용.

1. Max pooling 수행 → 각 영역의 최댓값과 **위치** 기록.
2. 네트워크의 나머지 부분에서 특징 처리.
3. 대응하는 unpooling 층에서 저장된 위치에 값을 배치.
4. 나머지 위치는 0.

기억하는 것은 최댓값의 위치.

중간 층을 거친 값이 그 위치에 놓이며, pooling 이전의 모든 값이 복원되는 것은 아님.

Learnable Upsampling — Transposed Convolution (Slide 47–58)

먼저 일반 convolution 복습:

- **Stride 1**: 입력에서 한 칸 이동 → 출력에서 한 칸 이동.
- **Stride 2**: 입력에서 두 칸 이동 → 출력에서 한 칸 이동.
- Strided convolution을 **학습 가능한 downsampling**으로 해석할 수 있음.

Transposed convolution은 입력값으로 필터를 가중해 출력 영역에 펼침.

- 입력의 각 값 × 필터 → 출력의 해당 영역에 배치.
- 입력에서 한 칸 이동할 때, 출력에서는 stride만큼 이동.
- 여러 필터 결과가 **겹치는 위치는 더함**.
- 필터가 학습되므로 **학습 가능한 upsampling**.

1차원 예시

입력 `[a, b]`, 필터 `[x, y, z]`. 슬라이드처럼 두 칸 간격으로 펼치면:

⇒ `az + bx`는 두 입력에서 펼친 결과가 겹쳐 합산된 부분.

Fully Convolutional 구조 정리

- **줄일 때**: pooling 또는 strided convolution.
- **키울 때**: unpooling 또는 strided transposed convolution.
- 마지막에는 원래 이미지 위치에 대응하는 클래스 예측을 생성.

고해상도에서 모든 계산을 수행하는 비용을 줄이면서, 픽셀별 출력을 만드는 흐름.

U-Net — 고해상도 특징을 다시 연결

Downsampling phase

- 공간 해상도를 줄임.
- 더 넓은 영역의 문맥을 볼 수 있음.
- 세밀한 공간 정보는 손실됨.

Upsampling phase

- 해상도를 키워 고해상도 예측을 생성.

두 경로 사이의 연결

- Downsampling 중 얻은 **고해상도 feature map을 저장**.
- 대응하는 upsampling 단계로 전달.

- 두 feature map을 **concatenate**해서 사용.

⇒ 넓은 문맥을 담은 특징과 세밀한 위치 정보를 함께 활용.

필기 보완: 슬라이드의 연산은 `concatenate`. 필기의 `F_enc + F_dec`는 두 특징을 합쳐 쓴다는 의미로 이해하고, 실제 연산은 **채널 방향 연결**로 구분해야 함.

Semantic Segmentation 정리

Sliding Window → 전체 이미지의 특징 공유 → downsampling·upsampling → U-Net의 고해상도 특징 연결

- 출력은 **픽셀별 클래스**.
- 같은 클래스의 개체를 따로 구분하지 않음.

2.2. Object Detection — 객체의 종류와 위치 찾기

단일 객체 — Classification + Localization

CNN에서 추출한 특징으로 두 가지를 함께 예측.

출력	예측 내용	슬라이드의 손실
클래스 점수	고양이, 개, 자동차 등	Softmax loss
Box 좌표	x, y, w, h	L2 loss

- 위치 추정을 **회귀 문제**로 처리.
- 분류 손실과 위치 손실을 더하는 **multitask loss**.

여러 객체 — 출력 개수가 달라지는 문제

- 객체 하나 → box 좌표 4개.
- 객체 세 개 → box 좌표 12개.
- 이미지마다 객체 수가 다름 → 필요한 출력 개수도 달라짐.

첫 번째 접근: 여러 crop을 CNN으로 분류.

- 각 crop이 `Dog / Cat / Background` 중 무엇인지 판단.
- 위치뿐 아니라 **크기·가로세로 비율**도 바꿔 가며 검사해야 함.

⇒ 검사할 crop 수가 너무 많아 계산 비용이 큼.

Region Proposals — 유망한 영역부터 고르기

객체가 있을 법한 영역을 먼저 추린 뒤, 그 영역을 자세히 검사.

- **Selective Search**: 객체를 포함할 가능성이 있는 영역들을 제안.
- 슬라이드 예시: CPU에서 수 초 내 **약 2,000개 proposal** 생성.
- Proposal은 객체 후보 영역이며, 최종 클래스와 box를 확정한 결과는 아님.

"Slow" R-CNN

후보 영역마다 CNN을 따로 실행.

1. Proposal 방법으로 **약 2,000개 RoI** 추출.
2. 각 영역을 **224 × 224**로 변형.
3. 각 영역을 ConvNet에 입력.
4. **SVM**으로 클래스 분류.
5. **Bounding-box regression**으로 proposal 위치 보정.

문제: 이미지 하나에 약 2,000번의 독립적인 CNN forward pass.

⇒ 개선 아이디어: **전체 이미지에서 CNN 특징을 먼저 구하고, feature map에서 영역을 잘라 쓰자.**

Fast R-CNN

CNN 특징을 여러 RoI가 공유.

1. 전체 이미지 → **backbone CNN**.
2. 이미지 전체의 feature map 생성.
3. 외부 proposal 방법에서 받은 RoI에 맞춰 **특징을 crop + resize**.
4. RoI별 네트워크에서 **클래스와 box offset** 예측.

R-CNN	Fast R-CNN
원본 이미지에서 영역을 자른 뒤 CNN	전체 이미지를 CNN에 넣은 뒤 특징을 자름
영역마다 CNN 계산 반복	backbone 특징 공유
SVM 분류	Linear + Softmax 분류

Backbone 예시: **AlexNet, VGG, ResNet**.

Region Proposal Network — Proposal도 학습하기

이미지 특징에서 객체 후보 영역을 직접 예측.

① 각 feature map 위치에 anchor box를 둠

- 각 위치를 기준으로 미리 정한 크기의 box를 가정.
- 실제로는 위치마다 **K개 anchor** 사용.

② 각 anchor의 objectness 예측

- “어떤 클래스인가?”에 앞서 **객체가 있는가?**를 판단.
- 슬라이드의 출력 표현:

③ Box 보정값 예측

- 객체가 있는 anchor에 대해 정답 box 방향으로 보정.
- Anchor 하나당 좌표 보정값 4개.

④ Proposal 선택

- 전체 후보 수:
- Objectness 점수로 정렬 → **상위 약 300개**를 proposal로 사용.

RPN의 역할: 객체 후보 제안.

Single-Stage Detectors — YOLO / SSD / RetinaNet

격자 위치에서 box와 클래스 정보를 직접 예측.

- 이미지를 **7 × 7 grid**로 나눔.
- Grid cell마다 **B개 base box**를 가정.
- Box마다 **좌표 보정값 4개 + confidence 1개**.
- Cell마다 **C개 클래스 점수**.
- 슬라이드 예시에서는 **B = 3**.
- C는 해당 도식에서 배경을 포함한 클래스 수로 설명.
- 형태는 RPN과 비슷하지만 **클래스 정보까지 예측**.

YOLO — Grid별 예측에서 최종 탐지까지

YOLO = You Only Look Once, 실시간 객체 탐지 방식으로 소개.

- 이미지를 **S × S grid**로 나눔.
- 각 cell을 기준으로 **B개 bounding box** 예측.
- Box 좌표, 객체 존재 점수, 클래스 점수를 출력.
- 이어지는 예시에서는 **B = 2**.

“왜 B개인가?”

- 한 cell에서 여러 box 후보를 예측하는 설정.
- 전체 box 후보 수:

그림 흐름

격자별 예측 → 많은 box 후보와 서로 다른 객체 점수 → 최종 box와 클래스 표시.

슬라이드 결과 예시: **개·자전거·자동차**를 각각 탐지.

DETR — Transformer로 객체 집합 예측

CNN 특징 → Transformer encoder–decoder → 클래스와 box의 집합

- Anchor 없이 box 좌표를 직접 예측.
- Anchor를 기준으로 한 box 보정값을 예측하는 방식에서 벗어남.
- 예측 집합과 정답 집합을 **bipartite matching**으로 연결해 학습.

구조

1. **Backbone CNN**: 이미지 특징 추출.
2. **Positional encoding + Encoder**: 위치 정보를 포함해 특징 처리.
3. **Object queries + Decoder**: 여러 객체 예측을 생성.
4. **Prediction heads**: 각 출력에서 클래스와 box 예측.
5. 객체와 대응하지 않는 출력은 **no object**로 처리.

Object query

- 학습 가능한 벡터.
- Decoder가 객체 예측을 만드는 데 사용하는 입력.
- Query 자체가 미리 정해진 bounding box는 아님.

Bipartite matching이 필요한 이유

- 정답 객체는 순서가 정해져 있지 않은 집합.

- 예측도 여러 개이므로, **어떤 예측을 어떤 정답과 비교할지** 결정해야 함.
- Matching 후 대응하는 클래스·box를 학습.

필기 보완: 학습에 필요한 정답은 **클래스와 box**. 추론 시 클래스와 위치는 모델이 예측.

2.3. Instance Segmentation — 객체별 픽셀 영역 구분

Faster R-CNN → Mask R-CNN

객체 탐지로 각 객체의 box를 찾은 뒤, **RoI마다 mask를 예측하는 분기**를 추가.

- 기존 출력: **클래스 + bounding box**.
- 추가 출력: **객체의 픽셀 마스크**.
- 작은 mask network가 RoI마다 **28 × 28 binary mask** 예측.

⇒ 같은 클래스에 속한 객체들도 서로 다른 마스크를 가짐.

Mask R-CNN 구조

이미지 → CNN + RPN → RoIAlign → 출력 분기

분기	슬라이드의 출력
분류	C개 클래스 점수
Box 예측	클래스별 좌표, 4C
Mask 예측	클래스별 마스크, C × 28 × 28

- RoIAlign으로 얻는 특징 예시: **256 × 14 × 14**.
- Mask 분기는 convolution을 거쳐 공간 정보를 유지하며 예측.
- 최종 mask 출력:

RoI별로, C개 클래스 각각에 대한 마스크를 예측하는 구성.

학습 target과 결과

- **Mask training target**: 해당 RoI 안에서 객체 영역을 나타내는 이진 마스크.
- 서로 다른 RoI에 대응하는 마스크 예시를 제시.
- 결과 그림: 여러 객체를 각각의 box와 mask로 구분.
- **Pose 추정에도 확장 가능**한 결과를 소개.

구현 프레임워크

강의에서 소개한 구현:

- **TensorFlow Detection API**
  - Faster R-CNN, SSD, R-FCN, Mask R-CNN 등.
- **Detectron2 — PyTorch**
  - Mask R-CNN, RetinaNet, Faster R-CNN, RPN, Fast R-CNN 등.

사전학습 모델을 가져와 자신의 데이터셋에 fine-tuning하는 활용 방식을 안내.

Computer Vision Tasks 복습

이미지 분류 → 픽셀 분류 → 객체별 box → 객체별 mask

과제를 구분할 때 확인할 것:

- **어느 위치인지** 출력하는가?
- **같은 클래스의 개체를 따로 구분**하는가?
- 위치 표현이 **box인가, 픽셀 마스크인가?**

---

3. Visualization & Understanding

3.1. Model Layers Visualization — 모델이 학습한 특징 보기

선형 분류기의 가중치 시각화

선형 분류기의 클래스별 가중치를 이미지 모양으로 배열.

- 클래스마다 서로 다른 가중치 패턴을 확인.
- 슬라이드에서는 비행기·자동차·새 등 클래스별 가중치 이미지를 제시.
- 입력과 클래스별 가중치의 관계를 **시각적인 템플릿 관점**에서 살펴봄.

CNN 첫 층의 필터 시각화

첫 convolution 층은 RGB 입력을 직접 받으므로 필터를 이미지로 볼 수 있음.

모델	첫 층 필터 크기
AlexNet	64 × 3 × 11 × 11

모델	첫 층 필터 크기
ResNet-18	64 × 3 × 7 × 7
ResNet-101	64 × 3 × 7 × 7
DenseNet-121	64 × 3 × 7 × 7

- 첫 숫자 **64**: 필터 개수.
- **3**: RGB 채널.
- 나머지: 필터의 공간 크기.
- 그림에서 방향성 있는 경계와 색상 변화 패턴을 관찰할 수 있음.

3.2. Saliency Maps — 어떤 입력 픽셀에 민감한가?

Backpropagation으로 계산

특정 클래스 점수가 입력 픽셀 변화에 얼마나 민감한지 확인.

1. 이미지를 넣어 forward pass.
2. 관심 클래스의 **정규화 전 점수** 선택.
3. 그 점수를 **입력 이미지에 대해 미분**.
4. 기울기의 절댓값 계산.
5. 각 픽셀에서 RGB 채널 중 최댓값 선택.

- `S_c`: 관심 클래스의 점수.
- `I`: 입력 이미지.
- 값이 큰 위치: 픽셀의 작은 변화에 클래스 점수가 민감한 위치.

3.3. CAM & Grad-CAM — 클래스별로 중요한 영역 보기

CAM — Class Activation Mapping

마지막 convolution 특징과 분류 가중치를 이용해 클래스별 공간 지도를 생성.

구조: **마지막 CNN feature map → Global Average Pooling → Fully Connected → class 점수**

기호:

- `H, W`: feature map의 공간 크기.

- `K`: 특징 채널 수.
- `C`: 클래스 수.
- `f`: 마지막 convolution 특징.
- `w_{k,c}`: 채널 k를 클래스 c의 점수에 반영하는 분류 가중치.

① 채널별 Global Average Pooling

공간 전체를 평균 → 채널마다 숫자 하나 → **K차원 벡터**.

② 클래스 점수 계산

합의 순서를 바꾸면, **각 위치의 클래스별 기여를 계산한 뒤 공간 평균한 형태**가 됨.

③ Class Activation Map 정의

- 클래스 c에 해당하는 가중치로 feature map 채널들을 가중합.
- 클래스마다 서로 다른 공간 지도 생성.
- 클래스 점수와의 관계:

⇒ **분류 점수가 이미지의 어느 영역에서 형성되는지** 공간적으로 살펴볼 수 있음.

CAM 예시와 한계

- 같은 건물 이미지에서도 `palace / dome / church` 등 클래스에 따라 강조 영역이 달라짐.
- `barbell` 클래스에서는 서로 다른 이미지의 바벨 관련 영역을 강조.

한계

- 제시된 **마지막 convolution → GAP → 선형 분류** 구조를 이용.

Grad-CAM — 기울기로 채널 가중치 구하기

선택한 층의 activation과 클래스 점수의 기울기를 이용.

① 시각화할 층 선택

공간 구조를 가진 activation을 대상으로 함.

② 관심 클래스 점수를 activation에 대해 미분

③ 기울기를 공간 평균 → 채널별 가중치

- 채널마다 평균 기울기 하나를 얻음.
- 관심 클래스에 대한 **채널의 가중치**로 사용.

④ Activation을 가중합하고 ReLU 적용

- 채널 가중치 × activation → 공간 지도.
- ReLU로 양의 값만 남김.

CAM과의 연결

방법	채널 가중치를 어디서 가져오나
CAM	마지막 선형 분류 층의 가중치
Grad-CAM	선택한 층에 대한 클래스 점수의 평균 기울기

Grad-CAM 예시

개와 고양이가 함께 있는 같은 이미지를 사용.

- 관심 클래스를 `Cat`으로 설정 → 고양이 쪽 강조.
- 관심 클래스를 `Dog`로 설정 → 개 쪽 강조.
- **같은 입력이어도 설명하려는 클래스에 따라 지도 변화.**

ViT 특징 시각화

- ViT에서도 입력 이미지와 공간적인 특징 시각화를 함께 제시.
- 여러 물체의 특징 지도와, 분류 확신도가 높은 이미지의 시각화 예시를 보여 줌.
- 이 슬라이드는 결과 예시 중심이며, 별도의 계산 절차나 수식은 제시하지 않음.

강조점

1. **Semantic segmentation은 픽셀별 클래스, instance segmentation은 객체별 마스크.** 같은 클래스의 개체를 구분하는지가 차이.
2. **분할의 흐름:** 주변 문맥 필요 → 특징 공유 → 해상도 축소·확대 → 고해상도 특징 재사용.
3. **U-Net은 encoder 특징을 decoder에 concatenate.** 공간 정보가 어떻게 전달되는지 확인.
4. **탐지의 흐름:** 많은 crop 검사 → proposal → CNN 특징 공유 → proposal도 학습 → grid별 직접 예측 → Transformer의 집합 예측.
5. **RPN은 objectness와 box 보정, 탐지기는 클래스까지 예측.**

6. **Mask R-CNN은 객체 탐지 구조에 RoI별 mask 예측을 추가.**
7. **Saliency는 입력 기울기, CAM은 분류 가중치, Grad-CAM은 activation에 대한 기울기**를 이용해 시각화.